

Boucle principale — attente *FRAMEREADY* et exécution de *SCENEACTUEL*

C'est le cœur battant du programme, celui qui tourne indéfiniment après l'initialisation.

Code complet

```
LD HL, FRAME_READY          ; Drapeau synchronisation 50Hz
BOUCLE_PRINCIPALE
LD  A, (HL)                  ; Charge la valeur du drapeau
AND A                        ; Teste si drapeau est égal à 0
JR  NZ, BOUCLE_PRINCIPALE    ; Si ce n'est pas 0, on boucle en attendant
SCENE_ACTUEL: CALL #0000
LD  HL, FRAME_READY
INC (HL)                     ; On remet le drapeau à 1 (bloquant)
JR  BOUCLE_PRINCIPALE        ; On repart pour un tour de boucle
```

Phase 1 : attente active de *FRAME_READY*

```
LD  A, (HL)
AND A
JR  NZ, BOUCLE_PRINCIPALE
```

HL pointe en permanence sur **FRAME_READY** (chargé une seule fois avant d'entrer dans la boucle, jamais rechargé à l'intérieur avant la phase 3). Le CPU relit cette valeur en boucle serrée — c'est une attente active (busy-wait), pas une vraie mise en veille : le Z80 tourne à plein régime, consommant des cycles sans rien faire d'utile, jusqu'à ce que **F_IM1_CALLBACK** détecte un vrai VBL et remette le drapeau à **0**. Ce n'est pas un gaspillage inutile pour autant — c'est le mécanisme le plus simple et le plus fiable pour garantir un timing exact sur Z80 sans OS ni scheduler : le programme reste disponible pour réagir à l'interruption IM1 (qui continue de tourner en tâche de fond, dispatchant les sous-tâches même pendant cette attente), tout en garantissant que **SCENE_ACTUEL** ne s'exécute jamais deux fois pour la même frame.

Phase 2 : exécution de la scène courante

```
SCENE_ACTUEL: CALL #0000
```

C'est le point de bascule le plus important architecturalement — un `CALL` dont l'opérande est réécrit dynamiquement. L'étiquette `SCENE_ACTUEL` marque l'adresse de l'octet `#0000` lui-même (l'opérande du `CALL`), pas une routine séparée. C'est exactement cet emplacement mémoire que `F_SCENE_CHARGER` modifie via `LD (SCENE_ACTUEL+1), HL` — le `+1` parce que l'opcode `CALL` occupe le premier octet, l'adresse cible les deux suivants.

Concrètement, selon la dernière scène chargée, ce `CALL` peut exécuter `F_SCENE_MENU`, `F_SCENE_JEU`, ou `F_SCENE_GAMEOVER` — sans que la boucle principale ait besoin de savoir laquelle. C'est cette indirection qui permet à `BOUCLE_PRINCIPALE` de rester totalement générique, immuable, peu importe le nombre de scènes que le jeu pourra accueillir.

C'est dans cet appel que s'enchaîne toute la logique de jeu : lecture clavier des raquettes, calcul des rebonds, déplacement de la balle sur ses deux axes, détection de but, puis affichage de chaque entité.

Phase 3 : libération du drapeau

```
LD    HL, FRAME_READY
INC    (HL)
JR     BOUCLE_PRINCIPALE
```

Une fois la scène traitée, `HL` est rechargé (il a potentiellement été modifié par `SCENE_ACTUEL`, puisque cette routine détruit `HL` selon les conventions établies pour la plupart des routines du projet). `INC (HL)` remet le drapeau à une valeur non nulle (`1`, sauf débordement improbable), ce qui relance immédiatement l'attente active de la phase 1.

Pourquoi INC plutôt que LD A,1

`INC (HL)` (11T) est légèrement plus rapide et plus court que `LD A,1 : LD (HL),A` (7+7=14T) pour ce cas précis, même si le gain ici est marginal vu que cette instruction ne s'exécute qu'une fois par frame (50 fois par seconde), pas dans une boucle serrée.

Le risque à surveiller : dépassement du budget de frame

Si `SCENE_ACTUEL` prend plus de ~20ms (le budget d'une frame à 50Hz), un nouveau VBL peut survenir **pendant** son exécution — `F_IM1_CALLBACK` remettrait alors `FRAME_READY` à `0` en plein milieu du traitement. Le `INC (HL)` qui suit masquerait silencieusement ce VBL manqué : au lieu de revivre une attente, la boucle repartirait immédiatement sur la frame suivante sans jamais avoir attendu le vrai rythme de l'écran — un frame-skip silencieux plutôt qu'un ralentissement visible.

C'est précisément pour rester sous ce budget que les optimisations suivantes prennent tout leur sens :

- Calcul incrémental de `VramAddr` plutôt que recalculé à chaque frame
- `LDI` au lieu de copies octet par octet pour les blits sprite

- Suppression des tables de lookup inutiles (`TabAdrLigne` , `CalculeBaseY`) au profit de constantes précalculées ou d'incréments directs
- Factorisation des blocs de code communs (paddle, balle) pour réduire la taille et le nombre d'instructions exécutées